Top 50 System Design Terminologies You Must Know

Tharun Kumar Reddy Polu · Follow
Published in InterviewNoodle · 9 min read · Jul 4, 2024

2.3K 22

Master the Essential Terms to Ace Your System Design Interviews with Explanations, Practical Examples, and Comprehensive Resources

Top 50 System Design Terminologies word cloud

System design interview performance is always a critical factor in validating whether a candidate can come up with scalable and efficient systems. Knowledge of major terminologies will definitely help in acing these. Below are the top 50 must-know system design interview terminologies that we will explain with definitions and working examples, along with additional resources for learning.

1. Scalability

- **Definition**: It is the ability of a system to support increased load by adding resources.
- **Example**: Addition of more servers to handle the increase in web traffic.
- **Learn More**: What is Scalability and How to Achieve it?

2. Load Balancer

- **Definition**: Dividing the incoming network traffic among multiple servers so that no one server processes a large amount of load.
- **Example**: Load balancing web traffic across multiple EC2 instances using the AWS Elastic Load Balancer(ELB) Service.
- **Learn More**: Understanding Load Balancer

3. Microservices

- **Definition:** It is an architectural pattern forcing the structuring of an application as a collection of loosely coupled services.
- **Example:** Breaking down a monolithic application into independent services responsible for user management, processing payments, and sending notifications.
- **Learn More**: What are Microservices?

4. CAP Theorem

- **Definition**: It states that at best, only two out of three guarantees can be gained in a distributed system: Consistency, Availability, and Partition Tolerance.
- **Example**: When to Trade Off Consistency for Availability — And Vice Versa — in Distributed Database Design.
- **Learn More**: Understanding CAP Theorem

5. Sharding

- **Definition**: It involves breaking down a large database into smaller pieces called shards for better management.
- **Example**: Sharding a user database based on geographic region.
- **Learn More**: Database Sharding Explained

6. Latency

- **Definition**: This gets defined as the time that it takes for data to travel from point A to point B.
- **Example**: Measuring the delay involved in message delivery through a chat application.
- **Learn More**: Latency explained!

7. Throughput

- **Definition**: A measure of the quantity of data a system processes in some timeframe
- **Example**: Requests processed by a web server in one second.
- **Learn More**: Throughput in Computer Networks

8. Cache

- **Definition**: Any hardware or software component that stores data to obviate future requests for the same data, serving It quickly.
- **Example**: Implementing Redis caching for repeated database queries.
- **Learn More**: Caching Explained

9. Content Delivery Network (CDN)

- **Definition**: A server system, geographically dispersed, that shows Web content to a user based on the geographical location from which he is accessing.
- **Example**: Using Cloudflare CDN for faster web page loading.
- **Learn More**: What is a CDN?

10. REST API

- **Definition**: a type of architectural style designed to build web services where data is accessed and manipulated using HTTP requests.
- **Example**: Designing the Social Media API by REST(Representational State Transfer) principles.
- **Learn More**: REST API Tutorial

11. GraphQL

- **Definition**: It is a language designed to query data, so it is much more powerful, efficient, and flexible than REST.
- **Example**: Using GraphQL to query user information in a single request.
- **Learn More**: GraphQL Introduction

12. ACID

- **Definition**: A set of properties ensuring reliable processing of database transactions. The properties are Atomicity, Consistency, Isolation, and Durability.
- **Example**: Ensuring that a banking transaction has ACID properties prevents corrupted data.
- **Learn More**: ACID Properties in Databases

13. BASE

- **Definition**: An alternate to ACID that emphasizes Availability and Partition tolerance over strict-Consistency. Basically Available, Soft state, Eventually consistent system.
- **Example**: Design of a highly available, eventually consistent NoSQL database.
- **Learn More**: BASE vs ACID

14. NoSQL

- **Definition**: A type of database designed to promote storage and retrieval of data modelled in ways other than the tabular relationships used in relational databases.
- **Example**: Using MongoDB for a document-based data store.
- **Learn More**: What is a NoSQL Database?

15. SQL

- **Definition:** It is the standard language used for storing, manipulating, and retrieving data in relational databases.
- **Example:** Writing SQL queries to get data back from a relational database.

- **Learn More**: SQL Tutorial

16. Database Indexing

- **Definition**: It is a data structure technique that allows quick searching and access to data from a database.
- **Example**: Create indexing on the column of User ID for searching speed enhancement.
- **Learn More**: Database Indexing

17. Replication

- **Definition**: A process of copying and maintaining database objects in a multitude of databases which make up a distributed database system.
- **Example**: It involves allowing a database to be highly available across different geographical locations using replication.
- **Learn More**: Database Replication

18. Failover

- **Definition**: A backup operational mode in which system component functions are taken over by other system components in case of loss of a primary system component.
- **Example**: Built-in automatic failovers to standby servers in the event of a server failure of your internet applications.
- **Learn More**: Failover Vs Disaster Recovery

19. API Gateway

- **Definition**: A server that sits at the front of an API, receiving API requests, applying throttling and security policies, and then forwarding them to back-end services.
- **Example**: Using AWS API Gateway to manage APIs.
- **Learn More**: What is an API Gateway?

20. Service Mesh

- **Definition**: A dedicated infrastructure layer for facilitating service-to-service communications between microservices.
- **Example**: Integrating Istio as a service mesh for the management of microservice interactions.
- **Learn More**: Introduction to Service Mesh

21. Serverless Computing

- **Definition**: A Cloud computing implementation that "dynamically allows for the allotment of machine resources by the cloud provider".
- **Example**: Run backend code without any server provisioning at your end using AWS Lambda.
- **Learn More**: What is Serverless Computing?

22. Event-Driven Architecture

- **Definition**: A software architecture paradigm encouraging the generation, detection, and consumption of, and the reaction to, events in general.
- **Example**: Design a system with event communications between microservices using Apache Kafka.
- **Learn More**: Event-Driven Architecture

23. Monolithic Architecture

- **Definition**: A software architecture wherein all the elements are fitted into a single application and run as a single service.
- **Example**: Old traditional enterprise applications built as a single, large unit.
- **Learn More**: Monolithic vs Microservices Architecture

24. Distributed Systems

- **Definition**: A model wherein components located on networked computers communicate with each other and coordinate their actions by passing messages.
- **Example**: Designing a distributed file system like Hadoop.

- **Learn More**: Introduction to Distributed Systems

25. Message Queue

- **Definition**: This method allows asynchronous, service-to-service communication in both serverless and microservices architectures.
- **Example**: Using RabbitMQ to queue messages between services.
- **Learn More**: Message Queues Explained

26. Pub/Sub Model

- **Definition:** A messaging pattern in which senders (publishers) publish messages so abstractly that any one of them can end up being accessed by recipients without the sender having to even know the identity of the destination receivers (subscribers).
- **Example:** A notification system that uses Google Cloud Pub/Sub.
- **Learn More**: Pub/Sub Messaging

27. Data Partitioning

- **Definition**: Division of a database into smaller, manageable parts.
- **Example**: Partitioning a table in a database by date to allow super-fast query execution.
- **Learn More**: Database Partitioning

28. Horizontal Scaling

- **Definition**: Increasing the capacity by adding more machines or nodes within a system.
- **Example**: Adding more web servers to handle an increasing volume of user traffic.
- **Learn More**: Horizontal vs Vertical Scaling

29. Vertical Scaling

- **Definition**: Upgrading an already existing machine with more power in the form of a CPU or RAM.

- **Example**: Upgrading the RAM of a server so that it can handle more requests all at once.
- **Learn More**: Horizontal vs Vertical Scaling

30. Rate Limiting

- **Definition**: It means controlling the rate of traffic that the network interface controller is sending or receiving.
- **Example**: Throttling an API to prevent abusive behaviour.
- **Learn More**: Understanding Rate Limiting

31. Circuit Breaker Pattern

- **Definition**: A design pattern used in modern software development, applied to detect failures and encapsulate the logic of preventing a failure from constantly recurring.
- **Example**: Handling failed remote service calls using a circuit breaker in a microservice architecture.
- **Learn More**: Circuit Breaker Pattern

32. Data Consistency

- **Definition**: Ensuring that data is the same across multiple instances and is not corrupted.
- **Example**: Maintaining the consistency of user data through multiple replicas of a database.
- **Learn More**: Data Consistency Models

33. Eventual Consistency

- **Definition**: A model of consistency used in distributed computing toward the goal of high availability, stating that updates to a system will eventually propagate and be reflected by all nodes.
- **Example**: Amazon DynamoDB provides an eventually consistent model for the read operation.
- **Learn More**: Eventual Consistency

34. Strong Consistency

- **Definition**: A consistency model ensuring every read gets the most recent write on a given unit of data.
- **Example**: Using strong consistency in a financial transaction system.
- **Learn More**: Strong Consistency

35. Containerization

- **Definition**: Basically, this is whenever an application and its dependencies are encapsulated into a container to be run on any computational environment.
- **Example**: Using Docker to containerize the applications for deployment in various environments such as dev, test, prod etc.
- **Learn More**: What is Containerization?

36. Kubernetes

- **Definition:** An open-source platform that automates the process of application container deployment, scaling, and operation.
- **Example:** Run and deploy containerized applications using Kubernetes.
- **Learn More**: Kubernetes Documentation

37. Autoscaling

- **Definition**: Automatically adjusting the number of computational resources based on the user load.
- **Example**: Utilizing AWS EC2 Auto Scaling feature to dynamically adjust the number of instances.
- **Learn More**: Auto Scaling explained

38. Multi-Tenancy

- **Definition:** Architecture where a single instance of a software application serves multiple consumers/customers.
- **Example:** SaaS applications, such as Salesforce, utilize multi-tenancy in their service provision toward their different categories of customers.
- **Learn More:** Single Tenancy Vs Multi-Tenancy?

39. Load Shedding

- **Definition**: Backing off some demands or degrading services to maintain the health of the overall system under high load.
- **Example**: This will turn off all non-essential services during times of peak traffic.
- **Learn More**: Load Shedding

40. Idempotence

- **Definition**: A property for some mathematical and computer-science operations stating that it has the same effect if repeated more times than once.
- **Example**: An HTTP DELETE request is idempotent.
- **Learn More**: Idempotence in APIs

41. Quorum

- **Definition**: The minimum number of votes needed to commit a distributed transaction.
- **Example**: Basically, quorum-based replication ensures that consistency exists in the distributed database.
- **Learn More**: Quorum Systems

42. Orchestration

- **Definition**: A pattern of service interaction where a central coordinator controls the interaction between services.
- **Example**: Using a workflow engine to manage some multi-step business process.
- **Learn More**: Orchestration

43. Choreography

- **Definition**: A service interaction pattern in which every service is self-contained and interacts with others through events; there will not be any coordinator or orchestrator.

- **Example**: Microservices communicating through an event bus in a choreography pattern.
- **Learn More**: Choreography vs. Orchestration

44. Service Registry

- **Definition**: A database that keeps track of instances of microservices.
- **Example**: Using the Eureka service registry in a microservice architecture.
- **Learn More**: Service Registry and Discovery

45. API Rate Limiting

- **Definition**: It means controlling how many requests a client can make against an API within a certain timeframe.
- **Example**: Limiting requests to an API to 100 per minute to prevent abuse.
- **Learn More**: API Rate Limiting

46. Data Warehouse

- **Definition:** A system that helps in the generation of reports and business data analytics; the hub of Business Intelligence.
- **Example:** Amazon Redshift can be implemented in data warehousing.
- **Learn More**: Understanding Data Warehouse?

47. Data Lake

- **Definition:** A system or repository where data is kept in native/raw format, generally as object blobs or files.
- **Example:** Petabyte scaling for storing and managing structured and unstructured data in a data lake.
- **Learn More**: Data Lake

48. OLAP

- **Definition:** Online Analytical Processing : The software category that allows the analysis of data kept in a database.

- **Example:** Use of the OLAP cubes for pointy analytical and arbitrary queries.
- **Learn More**: OLAP Explained

49. OLTP

- **Definition**: Online Transaction Processing: a class of systems that manage transaction-oriented applications.
- **Example**: Using OLTP systems for transaction data management, as in banking systems etc.
- **Learn More**: OLTP Explained

50. Big Data

- **Definition**: Large, complex data sets that cannot be efficiently managed by conventional data-processing software in the best of cases.
- **Example**: Analyzing social media interactions to predict fashion trends.
- **Learn More**: Introduction to Big Data

Keep in mind that it's all about continuous learning and practice as you go further in system design. You can work with the resources, get involved in the discussions, and practice these concepts in your projects. The resources and discussions will expose you to the vocabulary and usages of the concept.

Thanks for reading! Please share this guide with others if you, in any way, found it useful so they can also do these sets of exercises. If you have thoughts, questions, and/or resources, head down to our comments section.

Happy System Designing!

> *Connect with me on Linkedin, Github, Topmate, website to know more!*

InterviewNoodle Community

- 👏 Please clap for the story and follow the writer👉
- 📰 View more content on Coding and System Design Interviews
- 🔔 Subscribe to the InterviewNoodle Newsletter

System Design Concepts | System Design Interview | Distributed Systems | Software Architecture | Top50

Published in InterviewNoodle

3.4K Followers · Last published Jan 18, 2025

Follow

One stop learning portal for your next coding and system design interview.

Open in app ↗ Sign up Sign in

Medium Search Write

A software engineer documenting the personal tech experiences and exploring What, Why and How behind software engineering and large scale distributed systems.

Responses (22)

What are your thoughts?

Respond

JAGRITI BANSAL
Jul 14, 2024

Quite informative... found 2-3 points that i need to read in depth. Thanks!

63 2 replies Reply

Uservoxed
Jul 4, 2024

I can now officially add system design as a skill on my resume.....brb. :)

57 1 reply Reply

Sindi Sibanyoni
Jul 21, 2024

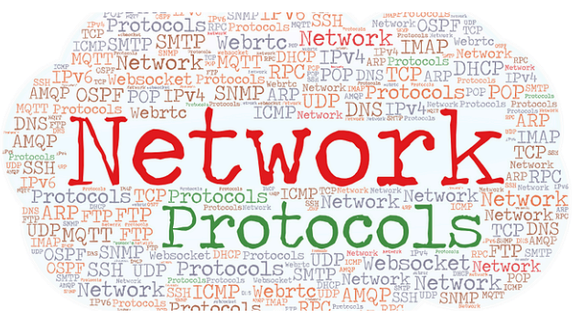
I found these resources very helpful as I work on building my vocabulary. Although I'm not in that specific field,

I'm familiarizing myself with a future venture I have in mind. I'll go through these a few more times. Thank you!!

56 1 reply Reply

See all responses

More from Tharun Kumar Reddy Polu and InterviewNoodle



In InterviewNoodle by Tharun Kumar Reddy Polu

Top 20 Network Protocols You Must Know in System Design

Master Key Network Protocols to Excel in System Design Interviews with Explanations,...

Aug 8, 2024 471 5

In InterviewNoodle by Freeze Francis

Google Phone Screen Interview Experience

Recently I had my 1st ever Google technical interview and it went pretty well. It was a...

Oct 13, 2022 506 7

In InterviewNoodle by Freeze Francis

Amazon SDE Interview Experience (On Campus)

Amazon visited our IIITB campus in 2017 for hiring both the intern and full-time positions....

Jul 24, 2021 622 3

Tharun Kumar Reddy Polu

How I Secured Multiple Graduate Assistantships in My First...

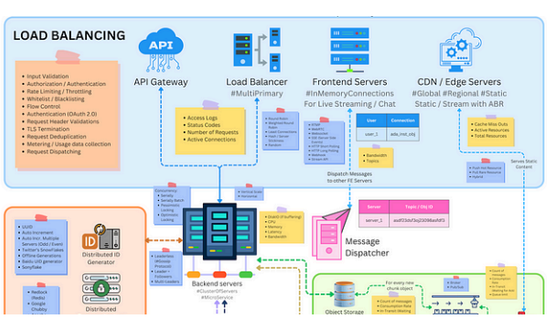
Crack cold emailing: strategies, effective techniques, and best practices—plus a...

Jul 19, 2024 5

See all from Tharun Kumar Reddy Polu

See all from InterviewNoodle

Recommended from Medium



In ByteByteGo System Design Alli... by Love Shar...

System Design Blueprint: The Ultimate Guide

Developing a robust, scalable, and efficient system can be daunting. However,...

Sep 18, 2023 9K 57

Talha Şahin

High-Level System Architecture of Booking.com

Take an in-depth look at the possible high-level architecture of Booking.com.

Jan 10, 2024 6.1K 48

Lists

Stories to Help You Grow as a Software Developer

19 stories · 1607 saves

General Coding Knowledge

20 stories · 1924 saves

Staff picks

817 stories · 1630 saves

Natural Language Processing

1955 stories · 1601 saves

In Coding Odyssey by Shivam Srivastava

Scenario Based Interview Question —3

Kafka and Java Based Question

Feb 11 55 2

In Stackademic by Oliver Foster

Why do many people not recommend using JWT?

My article is open to everyone; non-member readers can click this link to read the full text.

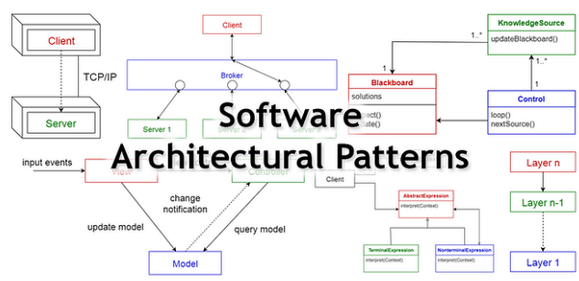
Sep 16, 2024 2.7K 76

Ramesh Fadatare

5 Microservices Design Patterns You Must Know in 2025

Here are five important microservices design patterns you should know in 2025, explained...

Jan 24 182 2



In TDS Archive by Vijini Mallawaarachchi

10 Common Software Architectural Patterns in a nutshell

Ever wondered how large enterprise scale systems are designed? Before major softwar...

Sep 4, 2017 41K 140

See more recommendations